AI Copilot

Full Proof of Concept (POC) Plan

CopilotKit • Agent Runtime • AG-UI • Frontend Tools • Zod • A2UI

Project: AI Employee Management Copilot

A full-stack learning and demonstration project showing how an AI agent can reason in the backend, invoke browser-side JavaScript tools, call backend business tools, stream events, and render validated dynamic UI.

Recommended approach

Build the application in milestones. First make the employee dashboard work without AI, then add CopilotKit, integrate the existing agent backend, verify frontend-tool dispatch, implement AG-UI streaming, and finally add a restricted A2UI renderer.

Planning document • Implementation details and API signatures must be checked against the exact versions selected for the POC.

1. Executive Summary

This POC is an employee management website with an embedded AI copilot. Users can ask questions about employees, populate forms through the assistant, search the database, request department analytics, and render supported UI components.

Core principle: the backend model does not directly execute browser JavaScript. The frontend registers a tool definition and handler; the agent requests the tool through a supported integration; the browser executes the handler and returns the result.

Learning objectives

- Understand the difference between a copilot, CopilotKit, CopilotKit Runtime, an AI agent, and an agent runtime.
- Connect a React frontend to a backend agent.
- Register frontend JavaScript functions as tools and verify how tool definitions reach the agent.
- Implement backend tools that query and modify business data.
- Use Zod to validate TypeScript-side tool arguments and structured inputs.
- Stream agent activity using AG-UI-compatible events.
- Render safe, structured dynamic UI using an A2UI-compatible protocol/catalog.
- Trace requests, tool calls, tool results, failures, and end-to-end latency.

User scenarios

- “Show me employees in Engineering.”
- “Open the registration form and fill in Arun’s details. Do not submit it yet.”
- “After I confirm, add this employee.”
- “Show department-wise employee counts as a chart.”
- “Generate a summary card and table for department performance.”

2. Concepts and Responsibilities

Concept	Responsibility
Copilot	An AI assistant embedded in an application to help users perform tasks.
CopilotKit	A framework for building copilot experiences and integrating the frontend with agent backends.
CopilotKit Runtime	The server-side integration component that handles copilot requests and connects to a configured agent backend; exact capabilities depend on the chosen version.
AI agent	Uses a model and instructions to decide what action to take.
Agent runtime	Executes and coordinates the agent workflow, model calls, tool calls, state and control flow.
Frontend tool	A registered browser-side capability whose JavaScript handler runs in the user's browser.
Backend tool	A server-side capability such as searching employees or creating a database record.
AG-UI	An event-based protocol for agent/frontend interactions, including messages, tool-call events and supported state updates.
A2UI	A declarative approach/protocol for describing UI components that a client renders using an approved catalog.
Zod	A TypeScript-first validation library for checking data against schemas.
Bash parameters	Arguments passed to a shell script, commonly accessed using $1, $2, $# and $@.

3. Recommended Technology Stack

Layer	Recommendation	Purpose
Frontend	React + TypeScript + Vite	Dashboard, form, table, copilot UI, frontend handlers and UI renderer.
Copilot integration	CopilotKit (pin chosen v2 release)	Copilot UI, frontend tool APIs and agent integration.
Runtime	CopilotKit Runtime	Routes/handles copilot requests and connects to the selected agent backend.
Agent backend	Existing agent SDK / ADK-Rust where feasible	Preserve current Bedrock/model setup and agent workflow.
Model	Existing Amazon Bedrock configuration	Model reasoning and tool selection.
Database	SQLite for local POC	Employee records and department summaries.
Validation	Zod on TypeScript side; corresponding validation on server	Validate tool arguments and request data at every trust boundary.
Protocol	AG-UI-compatible adapter	Translate the existing agent stream to the protocol/events required by the integration.
Dynamic UI	A2UI version pinned, or restricted renderer first	Render approved cards, tables, lists and charts.

Implementation note: the plan preserves the existing agent backend if possible. If the current agent is Rust-based, implement agent tools and the streaming adapter in Rust. If the working backend is FastAPI/Python, keep that service and add an adapter only where necessary.

4. High-Level Architecture

```
User
  |
  v
React frontend
  - Copilot UI
  - Employee dashboard, table and form
  - Frontend tool registry + handlers
  - Zod schemas
  - Approved dynamic UI renderer
  |
  | Copilot requests / streamed events
  v
CopilotKit Runtime
  - Request handling and agent integration
  |
  | AG-UI-compatible communication
  v
Agent backend / agent runtime
  - Agent SDK and model (e.g. Bedrock)
  - Agent instructions and workflow
  - Backend tool registry
  - Frontend-tool call coordination
  |
  v
Business services + SQLite
  - Employee CRUD
  - Department statistics
```

Frontend tool execution boundary: the agent requests a tool; the browser executes the registered handler. The backend must not try to call a browser-only function directly.

5. Proposed Repository Structure

```
employee-copilot/
■■■ frontend/
■   ■■■ src/
■       ■■■ app/                # App.tsx, providers.tsx
■       ■■■ components/         # Dashboard, form, table, chat, confirm dialog
■       ■■■ tools/              # Frontend tool registrations and schemas
■       ■■■ a2ui/               # Catalog, validator, renderer
■       ■■■ services/           # API client
■       ■■■ main.tsx
■■■ copilot-runtime/
■   ■■■ src/                    # Runtime configuration and entrypoint
■■■ agent-backend/
■   ■■■ app/
■   ■   ■■■ agent.*             # Agent initialization and workflow
■   ■   ■■■ tools/              # Employee and department tools
■   ■   ■■■ services/           # Business logic
■   ■   ■■■ schemas/            # Server-side validation
■   ■   ■■■ streaming/           # AG-UI adapter
■■■ database/
■   ■■■ schema.sql
■   ■■■ seed.sql
■■■ tests/
■   ■■■ frontend-tools/
■   ■■■ backend-tools/
■   ■■■ streaming/
■   ■■■ integration/
■■■ .env.example
■■■ docker-compose.yml
■■■ README.md
```

This is a logical target layout, not a requirement to use Python. Replace agent-backend file extensions and runtime details to match the existing Rust or Python service. Do not create a second agent service unless there is a specific integration need.

6. Milestone 1 — Build the Normal Employee App

Frontend screens

- **Dashboard:** total employee count, department counts, recent hires and simple analytics.
- **Employee directory:** searchable/filterable table with view and edit actions.
- **Employee form:** name, email, department, designation and joining date.
- **Confirmation dialog:** shows pending changes before a write operation.

Database schema

```
CREATE TABLE employees (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    department TEXT NOT NULL,
    designation TEXT NOT NULL,
    joining_date TEXT NOT NULL
);
```

REST endpoints

Endpoint	Purpose
GET /api/employees	List employees; support filters/search.
GET /api/employees/{id}	Retrieve one employee.
POST /api/employees	Create an employee.
PATCH /api/employees/{id}	Update an employee.
DELETE /api/employees/{id}	Delete an employee.
GET /api/departments/summary	Return department-level statistics.

Acceptance test: create an employee manually, see the row in the table, refresh the browser, and confirm the row remains in SQLite.

7. Milestone 2 — Connect CopilotKit and the Agent

1. Install and pin the chosen CopilotKit packages and version.
2. Add a chat/sidebar component to the React application.
3. Configure the CopilotKit Runtime endpoint.
4. Connect the Runtime to the existing agent backend using the supported integration.
5. Confirm the agent can receive a user message and stream a response.
6. Log request IDs/session IDs and errors without logging secrets.

```
User message
   -> React copilot UI
   -> CopilotKit Runtime
   -> Agent backend
   -> Model
   -> Agent response/stream
   -> Runtime
   -> React UI
```

Compatibility gate: confirm the installed Runtime's exact agent API. A custom FastAPI endpoint emitting AG-UI events is not automatically interchangeable with every CopilotKit Runtime adapter. Verify endpoint contracts and version-specific APIs before finalizing code.

Acceptance test: ask a basic employee-related question and observe a streamed response plus correlated frontend/backend logs.

8. Milestone 3 — Frontend Tools

Initial frontend tool set

Tool	Expected browser-side behavior
open_employee_form	Open the employee registration form.
fill_employee_form	Populate React form state.
filter_employee_table	Apply a department or search filter.
show_employee_details	Open the details panel for an employee.
navigate_to_page	Navigate only to an approved application route.

Illustrative tool registration

```
useFrontendTool({
  name: "fill_employee_form",
  description: "Fill the employee registration form",
  parameters: z.object({
    name: z.string(),
    email: z.string().email(),
    department: z.enum([
      "Engineering", "HR", "Finance", "Sales"
    ]),
    designation: z.string()
  }),
  handler: async (args) => {
    setEmployeeForm(args);
    return { success: true, message: "Form populated" };
  }
});
```

This is illustrative pseudocode. Verify the exact import path, hook name, parameters shape and handler contract against the selected CopilotKit release.

Frontend-tool discovery and execution lifecycle

1. Frontend registers the tool name, description, parameter schema and local handler.
2. The supported frontend-agent integration makes the tool definition available to the agent workflow.
3. The backend agent exposes the definition to the model as a callable tool.
4. The model requests the tool with structured arguments.
5. The tool-call event is relayed to the frontend.
6. The browser's tool registry dispatches the request to the registered handler.
7. The handler executes locally and returns a result to the agent.

Critical integration risk: if the custom agent framework does not automatically receive and expose frontend tool definitions, implement an adapter. It must pass definitions into model tool-calling, emit the proper frontend tool-call event, wait for the result, and resume the workflow.

Acceptance test: ask the agent to open and fill the form without submitting. Confirm the browser state changes and the database remains unchanged.

9. Milestone 4 — Backend Tools and Business Logic

Backend tool	Responsibility
search_employees	Search by name, department or query.
get_employee_details	Fetch a record by ID.
get_department_summary	Calculate counts/statistics.
create_employee	Validate and persist a new employee.
update_employee	Validate and update an existing record.
delete_employee	Delete only after the required authorization/confirmation.

Zod schema example (TypeScript boundary)

```
import { z } from "zod";

export const EmployeeSchema = z.object({
  name: z.string().min(1),
  email: z.string().email(),
  department: z.enum([
    "Engineering", "HR", "Finance", "Sales"
  ]),
  designation: z.string().min(1),
  joining_date: z.string()
});

export type Employee = z.infer<typeof EmployeeSchema>;
```

Zod validates data at the TypeScript boundary. If the agent backend is Python or Rust, implement equivalent validation there too; a frontend Zod schema does not enforce server-side rules.

Confirmation workflow

1. Agent gathers missing information.
2. Agent requests frontend tools to open and fill the form.
3. Frontend displays the populated form and pending values.
4. User reviews and confirms the write.
5. Application sends an authorized write request to the backend.
6. Backend validates permissions and data, then writes to SQLite.
7. Frontend refreshes the table and the agent reports the result.

Do not treat a model instruction as an authorization control. Enforce approval and permission checks at the server boundary for sensitive operations.

Acceptance test: search and create records through backend tools; reject invalid data; verify unconfirmed writes cannot reach the database.

10. Milestone 5 — AG-UI Streaming

AG-UI is an event-based protocol for agent/frontend interactions. It is distinct from the agent runtime and from A2UI's declarative UI descriptions.

Event family	Purpose
RUN_STARTED / RUN_FINISHED	Mark an agent run's lifecycle.
TEXT_MESSAGE_START / CONTENT / END	Stream assistant message content.
TOOL_CALL_START / ARGS / END	Represent tool-call information.
STATE_SNAPSHOT / STATE_DELTA	Communicate supported state updates.
RUN_ERROR	Report run errors.

Use the exact event names, payload schemas and sequence required by the selected AG-UI SDK/version. Not every workflow emits every event.

Adapter flow

```
Existing model stream
       |
       v
Existing agent SDK
       |
       v
AG-UI adapter
       |
       v
Compatible streaming endpoint
       |
       v
CopilotKit integration / frontend
```

- Preserve the current Bedrock stream wherever possible.
- Translate existing agent events into supported AG-UI events.
- Test event ordering, message IDs, run completion and errors.
- Test tool-call arguments, results, thread/session continuity and disconnect handling.
- Support cancellation/timeouts and prevent stale tool results from being applied.

Acceptance test: text streams incrementally; tool calls and results are visible; error and completion events are handled correctly.

11. Milestone 6 — A2UI and Dynamic UI

The agent should request structured UI descriptions rather than generating arbitrary HTML or JavaScript. The client validates these descriptions and renders them from a restricted catalog.

Example use case

User asks: “Show department-wise employee counts as a chart and summary card.” The agent retrieves the actual counts and returns a validated UI description that the frontend renders.

```
Illustrative application-level structure (not an A2UI protocol message):
{
  "type": "department_summary",
  "title": "Employee Distribution",
  "components": [
    {
      "type": "summary_card",
      "title": "Total Employees",
      "value": 20
    },
    {
```

```
      "type": "bar_chart",
      "data": [
        {"department": "Engineering", "count": 8},
        {"department": "HR", "count": 4},
        {"department": "Finance", "count": 5},
        {"department": "Sales", "count": 3}
      ]
    }
  ]
}
```

Approved UI catalog

Component	Allowed purpose
Text	Display text.
Card	Show a summary or metric.
Table	Display structured records.
BarChart	Display numeric comparisons.
List	Display a list of items.
Button	Trigger a specifically approved action.

- Pin a specific A2UI version and follow its real message envelope and component catalog.
- Validate messages before rendering and reject unknown component types/properties.
- Keep event transport and UI-description protocol separate.
- Never execute model-generated JavaScript or arbitrary HTML.
- Start with a restricted custom renderer if needed, then replace its message contract with the actual A2UI format.

Acceptance test: the agent returns a valid UI description and the frontend renders the requested card/table/chart using approved components.

12. Milestone 7 — Observability

Trace stage	Capture
Request received	Run ID, session/thread ID, timestamp.
Model step	Latency, selected tool metadata and status; avoid unnecessary sensitive prompt logging.
Tool call	Tool name, validated arguments, execution target and status.
Tool result	Success/failure, duration and safe result metadata.
Final response	Completion status, total duration and errors.

Use traces to diagnose whether a tool was not registered, not exposed to the model, selected incorrectly, not relayed to the browser, failed in the handler, or returned no result to the agent.

13. Milestone 8 — Security and Reliability

Risk	Mitigation
Invalid tool arguments	Validate at the receiving boundary.
Unauthorized employee access	Authenticate and authorize on the server.
Accidental writes	Require explicit confirmation for sensitive operations.
Arbitrary generated code	Never execute model-generated code.
Unsafe dynamic UI	Use a strict catalog and validate component properties.
Duplicate write calls	Use idempotency controls where appropriate.
Stream disconnects	Handle cancellation, reconnects and failed runs.
Sensitive logs	Redact personal data, credentials and secrets.

Use fictional employee data for the POC. Store model credentials only in server-side environment/configuration and never in browser-exposed variables.

14. Local Development and Deployment

Suggested local services

Service	Example local address
React frontend	http://localhost:5173
CopilotKit Runtime	http://localhost:3001
Agent backend	http://localhost:8000
Employee API	http://localhost:8000/api
SQLite	Local database file

These are proposed ports, not verified endpoints. Preserve existing working ports/routes where practical and adapt the Runtime configuration to the installed version.

Example environment template

```
# Frontend
VITE_API_BASE_URL=http://localhost:8000

# CopilotKit Runtime
AGENT_BACKEND_URL=http://localhost:8000

# Agent backend
DATABASE_URL=sqlite:///./employees.db
AWS_REGION=your-region
```

Use the existing AWS identity/Bedrock configuration. Never place AWS secret keys or other credentials in VITE_ variables. Once local integration passes, add Docker Compose, health checks, logs and a persistent database volume.

15. Testing Strategy and Definition of Done

- Dashboard and CRUD work without AI.
- Copilot connects to the backend agent and streams text.
- Frontend tool definitions reach the model/tool-calling layer.

- A browser-side handler executes locally and its result returns to the agent.
- Backend tools read and write database records.
- Invalid inputs are rejected at the server boundary.
- Unconfirmed writes are blocked.
- AG-UI event ordering, errors, disconnects and completion are tested.
- Dynamic UI descriptions are validated and rendered from an approved catalog.
- Logs correlate each run and tool call without exposing secrets.

End-to-end test scenarios

1. Ask a basic question and verify streamed text.
2. Ask the agent to filter the employee table and verify browser state changes.
3. Ask it to fill the form without submitting; verify no record is created.
4. Confirm submission and verify the new row in SQLite.
5. Send invalid arguments and verify rejection.
6. Interrupt a stream and verify clean termination.
7. Request a department chart and validate its UI description.
8. Attempt an unauthorized operation and verify the server rejects it.

16. Suggested Schedule

Day	Deliverable
1	Repository setup, dashboard, database and CRUD.
2	CopilotKit UI and Runtime/agent connection.
3	Frontend tool registration and Zod schemas.
4	Backend tools and confirmation workflow.
5	AG-UI adapter and streaming tests.
6	Tool tracing, error handling and security checks.
7–8	A2UI renderer/catalog and dynamic UI integration.
9–10	End-to-end tests, README and demo preparation.

Estimate: approximately 7–10 working days for one developer if the existing agent and Bedrock connection are usable. Protocol incompatibilities or unsupported frontend-tool lifecycles may add time.

17. Final Demo Script

1. User asks: “Register Arun Kumar in Engineering as a Software Engineer.”
2. The agent requests frontend tools; the form opens and fields populate in the browser.
3. The copilot displays the pending values and asks for confirmation.
4. After confirmation, the backend validates and inserts the employee record.
5. The employee table refreshes and the agent reports success.
6. User asks for department-wise employee distribution; the agent retrieves the data and returns a validated UI description.
7. The frontend renders a summary card and chart while the trace view shows model steps, tool calls, events and results.

18. Key Integration Risks and Decisions

- **Frontend-tool discovery:** verify how the installed CopilotKit frontend API exposes definitions to the selected backend agent; add an adapter if necessary.
- **Runtime compatibility:** verify the actual CopilotKit Runtime/agent integration API for the pinned package version.
- **AG-UI event format:** validate exact schemas and lifecycle behavior instead of assuming an arbitrary stream is compatible.
- **A2UI version:** pin a version and use its real protocol/catalog; do not label a custom JSON shape as A2UI.
- **Backend language:** keep the existing agent SDK where possible. The logical folder structure should not force a Python rewrite.
- **Write safety:** enforce authorization and confirmation server-side, not just in model instructions.

19. Recommended Build Order

1. Build the employee application and verify CRUD.
2. Connect CopilotKit UI, Runtime and the existing agent backend.
3. Implement one frontend tool end to end and trace its lifecycle.

4. Implement one backend database tool.
5. Complete the AG-UI streaming integration.
6. Add confirmation, validation, error handling and observability.
7. Add A2UI using a pinned protocol version and restricted catalog.
8. Run the acceptance tests, document setup and prepare the demo.

First technical task: verify the existing agent backend's stream format and the chosen CopilotKit version's supported integration. This decides whether a small adapter is enough or whether the tool-call lifecycle needs additional implementation.

20. Useful Official References

CopilotKit documentation: https://docs.copilotkit.ai/

CopilotKit repository: https://github.com/CopilotKit/CopilotKit

AG-UI documentation: https://docs.ag-ui.com/

AG-UI repository: https://github.com/ag-ui-protocol/ag-ui

A2UI documentation: https://a2ui.org/

A2UI repository: https://github.com/google/A2UI

Zod documentation: https://zod.dev/

Version note: package APIs and protocol specifications evolve. Before implementation, choose and pin compatible versions, then use the corresponding official documentation and schemas as the source of truth.